

Practica - 10 Trabajando con Archivos

Luis Gálvez

Tabla de contenidos

1 Proposito de la practica	1
1.1 Ejercicio 1: Limpiando un inventario forestal (I/O y Excepciones)	1
1.1.1 Desarrollo	2
1.2 Ejercicio 2: Analizando datos estructurados (Diccionarios y Listas)	3
1.2.1 Desarrollo	3
1.3 Preguntas	5
1.3.1 Desarrollo	5

1 Proposito de la practica

1.1 Ejercicio 1: Limpiando un inventario forestal (I/O y Excepciones)

A menudo recibimos datos de campo tomados manualmente que contienen errores tipográficos. Crea un script que realice lo siguiente:

1. Crea programáticamente un archivo llamado `arboles_crudos.txt` con el siguiente contenido (nota los errores intencionales en las latitudes y longitudes):

```
Especie,Latitud,Longitud
Roble,4.6097,-74.0817
Pino,4.6XYZ,-74.0900
Cedro,4.6150,-74.0850
Eucalipto,NULA,-74.0700
Nogal,4.6200,-74.0650
```

2. Escribe una función llamada `limpiar_inventario` que abra este archivo, lea línea por línea (omitiendo el encabezado) y utilice un bloque de manejo de excepciones (`try-except` / `tryCatch`) al intentar convertir las coordenadas a decimales (`float` / `numeric`).
3. Si la línea es válida, escríbela en un nuevo archivo llamado `arboles_limpios.csv`. Si falla, omítela pero imprime en consola una advertencia indicando qué especie tuvo el error.

1.1.1 Desarrollo

```
# -----
# 1. Crear el archivo original con errores intencionales

contenido = """Especie,Latitud,Longitud
Roble,4.6097,-74.0817
Pino,4.6XYZ,-74.0900
Cedro,4.6150,-74.0850
Eucalipto,NULA,-74.0700
Nogal,4.6200,-74.0650
"""

with open("arboles_crudos.txt", "w", encoding="utf-8") as archivo:
    archivo.write(contenido)

print("Archivo 'arboles_crudos.txt' creado correctamente.")

# -----
# 2. Función para limpiar el inventario

def limpiar_inventario():

    # Abrir archivo de entrada y archivo de salida
    with open("arboles_crudos.txt", "r", encoding="utf-8") as entrada, \
        open("arboles_limpios.csv", "w", encoding="utf-8") as salida:

        # Leer encabezado
        encabezado = entrada.readline()

        # Escribir encabezado en el archivo limpio
        salida.write(encabezado)

        # Leer línea por línea
        for linea in entrada:

            # Eliminar saltos de línea y separar columnas
            especie, latitud, longitud = linea.strip().split(",")

            try:
                # Intentar convertir coordenadas a float
                latitud = float(latitud)
                longitud = float(longitud)
```

```

        # Si no ocurre error, escribir línea limpia
        salida.write(f"{especie},{latitud},{longitud}\n")

    except ValueError:
        # Mostrar advertencia si las coordenadas son inválidas
        print(f"Advertencia: datos inválidos para la especie '{especie}'")

# -----
# 3. Ejecutar la limpieza

limpiar_inventario()

print("Proceso finalizado. Archivo 'arboles_limpios.csv' generado.")

```

Archivo 'arboles_crudos.txt' creado correctamente.
 Advertencia: datos inválidos para la especie 'Pino'
 Advertencia: datos inválidos para la especie 'Eucalipto'
 Proceso finalizado. Archivo 'arboles_limpios.csv' generado.

1.2 Ejercicio 2: Analizando datos estructurados (Diccionarios y Listas)

Vamos a tomar los datos limpios y pasarlos a la memoria estructurada de nuestro programa:

1. Crea una función `cargar_datos_espaciales` que lea el archivo `arboles_limpios.csv` que generaste en el Ejercicio 1.
2. Por cada línea, crea un Diccionario (o Lista nombrada en R) con las llaves `especie`, `latitud` y `longitud`.
3. Almacena todos esos diccionarios en una Lista o Arreglo principal.
4. Recorre esa lista final y calcula programáticamente **cuál es la latitud promedio** de los árboles válidos, imprimiendo el resultado en consola.

1.2.1 Desarrollo

```

# -----
# Función para cargar los datos espaciales

def cargar_datos_espaciales():

    # Lista principal donde se almacenarán los árboles
    arboles = []

    # Abrir archivo limpio
    with open("arboles_limpios.csv", "r", encoding="utf-8") as archivo:

```

```

        # Omitir encabezado
        next(archivo)

        # Leer línea por línea
        for linea in archivo:

            # Separar columnas
            especie, latitud, longitud = linea.strip().split(",")

            # Crear diccionario para cada árbol
            arbol = {
                "especie": especie,
                "latitud": float(latitud),
                "longitud": float(longitud)
            }

            # Agregar diccionario a la lista principal
            arboles.append(arbol)

        return arboles

# -----
# Cargar los datos

inventario = cargar_datos_espaciales()

# -----
# Mostrar estructura cargada

print("Datos cargados:\n")

for arbol in inventario:
    print(arbol)

# -----
# Calcular latitud promedio

suma_latitudes = 0

for arbol in inventario:
    suma_latitudes += arbol["latitud"]

latitud_promedio = suma_latitudes / len(inventario)

```

```
print("\nLatitud promedio de los árboles válidos:")
print(latitud_promedio)
```

Datos cargados:

```
{'especie': 'Roble', 'latitud': 4.6097, 'longitud': -74.0817}
{'especie': 'Cedro', 'latitud': 4.615, 'longitud': -74.085}
{'especie': 'Nogal', 'latitud': 4.62, 'longitud': -74.065}
```

Latitud promedio de los árboles válidos:
4.6149

1.3 Preguntas

1. **Sobre el Ejercicio 1 (Manejo de Excepciones):** Explica por qué procesar datos espaciales usando bloques `try...catch` (o `except`) es una mejor práctica que dejar que el intérprete arroje un error por defecto. ¿Qué pasaría con un archivo de 500,000 registros si el registro número 499,999 tiene un error tipográfico y no usaste manejo de excepciones?
2. **Sobre la Gestión de Recursos (I/O):** Compara cómo Python y Julia manejan el cierre automático de archivos (`with open(...)` as `f`: vs `open(...)` do `f`) frente al enfoque de R (`close()`). ¿Por qué es un riesgo de seguridad y rendimiento dejar conexiones de archivos abiertas en un flujo de geoprocesamiento continuo?
3. **Sobre la Limpieza de Datos:** Explica brevemente cuál es la función de los comandos `strip()` (o `trimws()`) y `split()` al momento de crear bases de datos a partir de archivos de texto plano. ¿Qué errores silenciosos evitan al momento de intentar convertir texto a números decimales?

1.3.1 Desarrollo

1. El manejo de excepciones mediante bloques `try...except` en Python o `try...catch` en otros lenguajes permite que un programa continúe ejecutándose incluso cuando encuentra datos corruptos o inesperados. En procesamiento espacial esto es fundamental porque los datos de campo suelen contener errores tipográficos, coordenadas incompletas o formatos inconsistentes. Si se deja que el intérprete arroje el error por defecto, la ejecución completa del programa se detiene inmediatamente al encontrar el primer dato inválido. En un archivo con 500,000 registros, si el registro 499,999 tuviera un error y no existiera manejo de excepciones, el procesamiento fallaría casi al final después de consumir tiempo, memoria y recursos computacionales durante toda la ejecución. Además, no se generarían los resultados finales ni se identificarían automáticamente los registros problemáticos. Con manejo de excepciones, el programa

puede omitir únicamente el dato defectuoso, registrar una advertencia y continuar procesando el resto de la información.

2. Python y Julia implementan mecanismos seguros de manejo de archivos mediante contextos automáticos (`with open(...)` as `f` en Python y `open(...)` do `f` en Julia). Estos enfoques garantizan que el archivo se cierre automáticamente incluso si ocurre un error durante la lectura o escritura. En R, aunque existen funciones equivalentes de alto nivel, muchas veces el programador debe cerrar manualmente las conexiones usando `close()`, lo que aumenta el riesgo de olvidar liberar recursos. Mantener archivos abiertos en procesos de geoprocesamiento continuo puede causar fugas de memoria, bloqueo de archivos, agotamiento del límite de conexiones del sistema operativo y corrupción de datos si múltiples procesos intentan acceder simultáneamente al mismo archivo. En flujos espaciales grandes, donde se leen cientos de capas, rásteres o tablas, esto también afecta el rendimiento porque el sistema mantiene recursos ocupados innecesariamente.
3. El comando `strip()` en Python o `trimws()` en R elimina espacios en blanco, saltos de línea y caracteres invisibles al inicio y al final de un texto. Esto es importante porque muchos archivos CSV o TXT contienen espacios accidentales que impiden convertir correctamente cadenas a números. Por ejemplo, " 4.6097 " puede generar problemas en ciertos contextos de validación o comparación textual. Por otro lado, `split()` divide una línea de texto usando un separador, normalmente una coma en archivos CSV, permitiendo separar cada columna individualmente. Ambos comandos son esenciales en la limpieza de datos porque evitan errores silenciosos como columnas mal separadas, coordenadas interpretadas como texto, fallos en conversiones a float o valores desplazados entre campos. Sin estas operaciones, una base de datos espacial puede contener coordenadas incorrectas o registros imposibles de procesar sin que el usuario detecte fácilmente el problema.